

Отчёт по третьему этапу внешнего курса Stepik

Продвинутые темы

Иванова Ангелина Олеговна

2026-05-08

Содержание I

- 1 Вводная часть
- 2 Выполнение лабораторной работы
- 3 Результаты

Section 1

Вводная часть

Цель работы

Целью данной работы является выполнение внешнего курса под названием «Введение в Linux». В третьем этапе мы подробно редактор vim, скрипты bash и различные возможности Linux.

- 1 Ознакомиться с теоретическим материалом

- 1 Ознакомиться с теоретическим материалом
- 2 Ответить на вопросы и выполнить задания для закрепления теоретического материала

Section 2

Выполнение лабораторной работы

Выполнение 3.1. Текстовый редактор vim

3.1 Текстовый редактор vim 12 из 12 шагов пройдено 7 из 7 баллов получено

Вы прошли больше 80% курса, оставьте отзыв

[Оставить отзыв](#) [Нет, спасибо](#)

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

✔ Отличное решение!

- ☐ "Esc"
- ☐ "Q"
- ☐ "q", затем "Enter"
- ☐ ":", затем "q"
- ☒ ":", затем "q", затем "Enter"

[Решить снова](#)

Вы получили: 1 балл

Моя реакция

Верно решили 47 439 учащихся

Из всех попыток **67%** верных

Рисунок 1: Выход из редактора vim

Выполнение 3.1. Текстовый редактор vim

3.1 Текстовый редактор vim 12 из 12 шагов пройдено 7 из 7 баллов, получено

При перемещении в *vit* "по словам" есть небольшая разница в том, используем мы маленькую (*w, e, b*) или большую (*W, E, B*) букву. Первые перемещают нас по "словам" (*word*), а вторые по "большим словам" (*WORD*). Посмотрите справку по этим перемещениям и разберитесь в чем заключается разница между *word* и *WORD*.

А для того, чтобы убедиться, что вы разобрались, отметьте ниже все верные утверждения про следующую строку

```
Strange_ TEXT is_here. 2=2 YES!
```

Примечание: во всех утверждениях имеется ввиду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Подсказка: чтобы вызвать **vim-справку** по, например, переименованию **»,** нужно открыть **vim** и ввести команду **:help »**. Вы попадете в место справки, где описано это переименование, а так как все переименования описаны рядом, то двигаясь по тексту вверх и вниз можно прочитать и про **»** и про **»**, самое главное, про **word** и **WORD**. Кроме того, можно вызвать сразу справку по термину **word** при помощи **:help word**. Чтобы вызвать справку, нужно ввести команду **:g**.

Выберите все подходящие ответы из списка

✓ Все правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Нажимаем только на W, нельзя переместить курсор на "
- ☒ После 10 нажатий на W курсор окажется там же, где он был после 10 нажатий на w
- ☐ Чтобы попасть в конец строки, нужно одинаковое число нажатий, что на W, что на w
- ☒ Чтобы попасть в конец строки, нужно совершить меньше нажатий на W, чем на w
- ☐ В этой строке 12 "слов" (word)
- ☐ Нажимаем только на w, нельзя переместить курсор на "

Результат снова

Рисунок 2: Использование w и W

Выполнение 3.1. Текстовый редактор vim

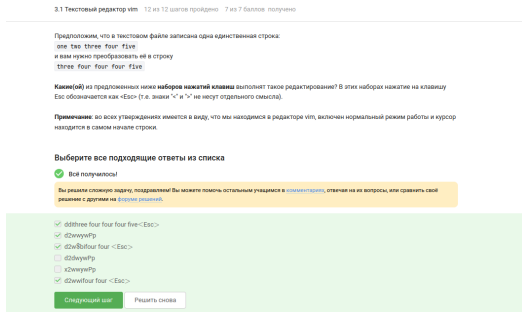


Рисунок 3: Изменение строки

Выполнение 3.1. Текстовый редактор vim

3.1 Текстовый редактор vim 12 из 12 шагов пройдено 7 из 7 баллов получено

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно только самое первое из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. включая ввод `:` в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак не нужно.

Напишите текст

✓ Хорошие новости, верно!

Следующий шаг Решить снова

Рисунок 4: Замена Windows на Linux

Выполнение 3.1. Текстовый редактор vim

3.1 Текстовый редактор vim 12 из 12 шагов пройдено 7 из 7 баллов получено

Мы совсем не рассказали вам про третий режим работы vim – режим **выделения (Visual)**. Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в vimtutor, который мы настоятельно рекомендуем вам для изучения vim!

Чтобы убедиться, что вы разобрались с этим режимом работы, отметьте, пожалуйста, **все верные** утверждения из списка ниже.

Подсказка: если вы не хотите проходить vimtutor целиком, то можете открыть его и поиском найти слово **"Visual"**. Вы попадете в задание, прохождение которого будет вполне достаточно, чтобы выполнить это задание.

Выберите все подходящие ответы из списка

☒ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Режим выделения открывается из нормального режима по нажатию "v"
- ☐ Режим выделения открывается из любого другого режима по нажатию "v"
- ☒ В режиме выделения можно использовать команды d (удалить) и у (скопировать)
- ☐ Чтобы выйти из режима выделения, нужно ввести :q
- ☒ В режиме выделения можно использовать команды перемещения (например, W, e, \$, и др.)
- ☒ Выйти из режима выделения можно, нажав клавишу Esc два раза

Следующий шаг

Решить снова

Рисунок 5: Режим Visual в vim

Выполнение 3.2. Скрипты на bash: основы

3.2 Скрипты на bash: основы 10 из 10 шагов пройдено 6 из 6 баллов получено

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попытаете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка

✔ Все получилось!

- ☐ Из наборов B и C
- ☐ Только из набора B
- ☐ Никакие команды появляться не будут
- ☐ Из наборов A и C
- ☒ Только из набора C

Следующий шаг Решить снова

Рисунок 6: История команд при вложенных оболочках

Выполнение 3.2. Скрипты на bash: основы

3.2 Скрипты на bash: основы 10 из 10 шагов пройдено 6 из 6 баллов получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#), [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash

cd /home/bi/
touch file1.txt
cd /home/bi/Desktop/
```

Как будет выглядеть абсолютный путь до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один вариант из списка

✔ Отличное решение!

- ☒ /home/bi/file1.txt
- ☐ Никак (файл file1.txt не будет существовать после завершения работы скрипта)
- ☐ /home/bi/Documents/file1.txt
- ☐ /home/bi/Desktop/file1.txt

[Решить снова](#)

Рисунок 7: Абсолютный путь созданного файла

Выполнение 3.2. Скрипты на bash: основы

3.2 Скрипты на `bash`: основы 10 из 10 шагов пройдено 6 из 6 баллов получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк **могут** быть именами переменных в `bash`? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Выберите все подходящие ответы из списка

Отлично!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ VARIABLE
- ☒ variable
- ☐ var i able
- ☒ __variable
- ☐ 123variable
- ☒ variable_123
- ☒ variable123

[Решить снова](#)

Рисунок 8: Допустимые имена переменных

Выполнение 3.2. Скрипты на bash: основы

3.2 Скрипты на bash: основы 10 из 10 шагов пройдено 6 из 6 баллов получено

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [arguments.sh](https://github.com/arguments.sh).

Напишите скрипт на `bash`, который принимает на вход два аргумента и выводит на экран строку следующего вида:

Arguments are: \$1=первый_аргумент \$2=второй_аргумент

Например, если ваш скрипт называется `./script.sh`, то при запуске его `./script.sh one two` на экране должно появиться:

Arguments are: \$1=one \$2=two

а при запуске `./script.sh three four` будет:

Arguments are: \$1=three \$2=four

Подсказка: в случае проблем с решением задачи, обратите внимание [на наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✔ Всё получилось!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2 var1=$1
3 var2=$2
4
5 echo "Arguments are: \${1}=${var1} \${2}=${var2}"
```

[Решить снова](#)

Рисунок 9: Скрипт вывода аргументов

Выполнение 3.3. Скрипты на bash: ветвления и циклы

3.3 Скрипты на bash: ветвления и циклы 9 из 9 шагов пройдено 10 из 10 баллов получено

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [branching1.sh](https://github.com/ArtemSeregin/branching1.sh).

Предположим, вы пишете скрипт на `bash` и хотите использовать в нем конструкцию `if` в следующем фрагменте:

```
if [[ ... ]]
then
    echo "True"
fi
```

Вы можете вписать вместо "*" (внутри `[[ ]]`) и не забудьте про пробелы после `[[` и перед `]]`) любое из перечисленных ниже условий. Однако мы просим вас выбрать только те из них, при которых `echo` напечатает на экран `True` вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные.

Например, условие `0 == 0` **подходит**, т.к. ноль всегда равен нулю вне зависимости от аргументов и переменных внутри скрипта и на экран будет напечатано `True`. В то же время условие `$var1 == 0` **не подходит**, так как в переменной `var1` как может быть записан ноль (тогда будет напечатано `True`), так его может и не быть (тогда ничего напечатано не будет).

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ \$ тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания \$ в некоторых видах заданий на Stepik.

Выберите все подходящие ответы из списка

✔ Все правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ ! (4 -le 3)
- ☒ \$# -ge 0
- ☒ -e \$0
- ☒ -n \$0
- ☒ -z ""
- ☒ 5 -ge 5

[Решить снова](#)

Рисунок 10: Всегда истинные условия

Выполнение 3.3. Скрипты на bash: ветвления и циклы

3.3 Скрипты на bash: ветвления и циклы 9 из 9 шагов пройдено 10 из 10 баллов получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [branching2.sh](#), [branching3.sh](#).

Посмотрите на фрагмент `bash`-скрипта:

```
if [[ $var -gt 5 ]]
then
    echo "one"
elif [[ $var -lt 3 ]]
then
    echo "two"
elif [[ $var -eq 4 ]]
then
    echo "three"
else
    echo "four"
fi
```

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную `var=3`, а затем запустили еще раз, но уже с `var=5`.

Выберите один вариант из списка

✔ Так точно!

- ☐ Сначала two, потом one
- ☐ Сначала four, потом one
- ☒ Сначала four, потом four
- ☐ Сначала one, потом two

[Решить снова](#)

Рисунок 11: Ветвление с числовыми сравнениями

Выполнение 3.3. Скрипты на bash: ветвления и циклы

3.3 Скрипты на `bash`: ветвления и циклы 9 из 9 шагов пройдено 10 из 10 баллов получено

Напишите скрипт на Bash, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории. В зависимости от значения числа нужно вывести разные сообщения.

Соответствие входа и выхода должно быть таким:

```
0 --> No students
1 --> 1 student
2 --> 2 students
3 --> 3 students
4 --> 4 students
5 or more --> A lot of students
```

Примечание а): выводить нужно только строку справа, т.е. " $\rightarrow$ " выводить не нужно.

Примечание б): в последней строке слово 'lot' с маленькой буквы!

Примечание 2: в этой и всех последующих задачах на написание скриптов, если не указано явно, что нужно проверить вход (например, что он будет именно числом и именно от 0 до бесконечности), то этого делать не нужно!

Пример №1: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 1` на экране должно появиться:

1 student

Пример №2: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 5` на экране должно появиться:

A lot of students

Подсказка: в случае проблем с решением задачи, обратите внимание на наши рекомендации по написанию скриптов

Напишите программу. Тестируется через stdin → stdout

✓ Хорошие новости, верно!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1#!/bin/bash
2
3if [[ $1 -eq 1 ]]; then
4    echo "$1 student"
5elif [[ $1 -gt 1 && $1 -le 4 ]]; then
6    echo "$1 students"
7elif [[ $1 -ge 5 ]]; then
8    echo "A lot of students"
9else
10    echo "No students"
11fi
12
```

[Решить снова](#)

Рисунок 12: Скрипт для подсчёта студентов

Выполнение 3.3. Скрипты на bash: ветвления и циклы

3.3 Скрипты на bash: ветвления и циклы 9 из 9 шагов пройдено 10 из 10 баллов получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [loops1.sh](#), [loops2.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
for str in a , b , c,d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то сколько раз на экран будет выведено слово "start", а сколько раз слово "finish"?

Выберите один вариант из списка

☒ Так точно!

☐ 5 раз "start" и ни разу "finish"

☐ 5 раз "start" и 2 раза "finish"

☒ 5 раз "start" и 4 раза "finish"

☐ 3 раза "start" и 3 раза "finish"

Рисунок 13: Цикл с continue

Выполнение 3.3. Скрипты на bash: ветвления и циклы

3.3 Скрипты на bash: ветвления и циклы 9 из 9 шагов пройдено 10 из 10 баллов получено

Напишите скрипт на bash, который будет определять в какую возрастную группу попадают пользователи. При запуске скрипт должен вывести сообщение **"enter your name:"** и ждать от пользователя ввода имени (используйте `read`, чтобы прочитать его). Когда имя введено, то скрипт должен написать **"enter your age:"** и ждать ввода возраста (опять нужен `read`). Когда возраст введен, скрипт пишет на экран **"<Имя>, your group is <группа>"**, где **<группа>** определяется на основе возраста по следующим правилам:

- младше либо равно 16: **"child"**,
- от 17 до 25 (включительно): **"youth"**,
- старше 25: **"adult"**.

После этого скрипт опять выводит сообщение **"enter your name:"** и всё начинается по новой (бесконечный цикл). Если в какой-то момент работы скрипта будет введено **пустое имя** или **возраст 0**, то скрипт должен написать на экран **"bye"** и закончить свою работу (выход из цикла).

Примеры корректной работы скрипта:

№1

```
./script.sh
enter your name:
Egor
enter your age:
16
Egor, your group is child
enter your name:
Elena
enter your age:
8
bye
```

№2:

```
./script.sh
enter your name:
Elena Petrovna
enter your age:
25
Elena Petrovna, your group is youth
enter your name:

bye
```

Подсказка: в случае проблем с решением задачи, обратите внимание на наши рекомендации по написанию скриптов.

Рисунок 14: Скрипт возрастной группы – условие

Выполнение 3.4. Скрипты на bash: разное

3.4 Скрипты на bash: разное 10 из 10 шагов пройдено 14 из 14 баллов получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [math1.sh](#), [math2.sh](#).

Какие(ая) из предложенных ниже инструкций увеличат значение переменной **a** на значение переменной **b**? Например, если в **a** было записано 10, а в **b** было 5, то в **a** должно записаться 15.

Выберите **все подходящие** варианты!

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ \$ тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания \$ в некоторых видах заданий на Stepik.

Подсказка: обратите особое внимание на **кавычки** и **пробелы**, они могут как принципиально изменить команду, так и ни на что не повлиять (в зависимости от команды и контекста)!

Выберите все подходящие ответы из списка

✔ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ $a = \$a + \b
- ☒ `let a = a + b`
- ☒ `let "a" = a + b`
- ☒ `let "a" += b`
- ☒ `let "a" = $a + $b`

[Решить снова](#)

Рисунок 16: Увеличение переменной

Выполнение 3.4. Скрипты на bash: разное

3.4 Скрипты на bash: разное 10 из 10 шагов пройдено 14 из 14 баллов получено

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [programs.sh](#).

Пусть вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
```

```
cd /home/b1/
echo "`pwd`"
```

Что в этом случае выведет команда `echo` на экран?

Выберите один вариант из списка

Отлично!

- ☐ 'pwd'
- ☐ /home/bi/Documents
- ☐ pwd
- ☐ Код возврата команды pwd (0 в случае успешного выполнения и не 0 в случае ошибок)
- ☒ /home/bi

[Решить снова](#)

Вы получили: 1 балл

Мои решения

Верно решил 35 241 учащийся

Из всех попыток **51%** верных

Рисунок 17: Вывод текущего каталога в скрипте

Выполнение 3.4. Скрипты на bash: разное

3.4 Скрипты на bash: разное 10 из 10 шагов пройдено 14 из 14 баллов получено

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции `if` при помощи `if "progname options arguments"` (действия внутри `if` выполняются, если программа закончилась с кодом 0). Однако это не всегда правда! Если запуск внешней программы выводит что-то в `stdout`, то в проверку `if` поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой `bash-script` с использованием, например, `if "pwd"`.

Однако как быть, если хочется все-таки запустить программу `progam`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если ее код возврата равен 0? Выберите **все верные** утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, используется именно косая кавычка ('), а не обычная (") или двойная (").

Выберите все подходящие ответы из списка

✓ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Ничего сделать нельзя
- ☒ Сначала запустить program, затем if [[\$? -eq 0]]
- ☐ if [['program' -eq 0]]
- ☒ if 'program' > some_file.txt
- ☐ Сначала var='program', затем if [[\$var -eq 0]]

[Решить снова](#)

Рисунок 18: Возврат внешней программы

Выполнение 3.4. Скрипты на bash: разное

3.4 Скрипты на bash: разное 10 из 10 шагов пройдено 14 из 14 баллов получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [functions1.sh](#), [functions2.sh](#).

Посмотрите на функцию из `bash`-скрипта:

```
counter () # takes one argument
{
  local let "c1+=$1"
  let "c2+=${1}*2"
}
```

Впишите в форму ниже **строку**, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте **после десяти вызовов** функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое 😊). В этом случае имеет смысл написать небольшой скрипт на `bash`, который проделает ровно то, что указано в задании и посимвольно сверит свой ответ с тем, что он выдает на экран.

Напишете текст

✔ Правильно, молодец!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форме решений](#).

counters are and 110

[Решить снова](#)

Рисунок 19: Локальные переменные в функции

Выполнение 3.4. Скрипты на bash: разное

3.4 Скрипты на bash: разное 10 из 10 шагов пройдено 14 из 14 баллов получено

Напишите скрипт на `Bash`, который будет искать наибольший общий делитель (`НОД`, `greatest common divisor`, `GCD`) двух чисел. При запуске ваш скрипт не должен ничего писать на экран, а просто ждать ввода двух натуральных чисел через пробел (для этого можно использовать `read` и указать ему две переменные – см. пример в викифрагменте). После ввода чисел скрипт считает их `НОД` и выводит на экран сообщение **"GCD is «числитное значение»"**, например, для чисел 15 и 25 это будет **"GCD is 5"**. После этого скрипт опять входит в режим ожидания двух натуральных чисел. Если в какой-то момент работы пользователь ввел вместо этого пустую строку, то нужно написать на экран **"bye"** и закончить свою работу.

Вычисление НОД несложно реализовать с помощью **алгоритма Евклида**. Вам нужно написать функцию `gcd`, которая принимает на вход два аргумента (назовем их **M** и **N**). Если аргументы равны, то мы нашли НОД – он равен **M** (или **N**), можно вывести соответствующее сообщение на экран (см. выше). Иначе нужно сравнить аргументы между собой. Если **M** больше **N**, то запускаем ту же функцию `gcd`, но в качестве первого аргумента передаем **(M-N)**, а в качестве второго **N**. Если же наоборот, **M** меньше **N**, то запускаем функцию `gcd` с первым аргументом **M**, а вторым **(N-M)**.

Пример корректной работы скрипта:

```
./script.sh
10 15
GCD is 5
7 3
GCD is 1
bye
```

Примечание: в вызове функции из себя самой нет ничего страшного или неправильного, т.ч. смело вызывайте `gcd` прямо внутри `gcd`!

Примечание 2: для завершения работы функции в произвольном месте, можно использовать инструкцию `return` (все инструкции функции после `return` выполняться не будут). В отличие от `exit` эта команда завершит только функцию, а не выполнение всего скрипта целиком. Однако в данной задаче можно обойтись и без использования `return`!

Подсказка: в случае проблем с решением задачи, обратите внимание [на наши рекомендации по написанию скриптов](#).

Рисунок 20: Задание на НОД

Выполнение 3.4. Скрипты на bash: разное

Напишите программу. Тестируется через stdin → stdout

✔ Здорово, всё верно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета

```

1 #!/bin/bash
2
3 while [ true ]
4 do
5     read n1 n2
6     if [ -z $n1 ] ; then
7         echo "bye"
8         break
9     else
10         gcd () {
11             remainder=1
12             if [ $n2 -eq 0 ]
13             then
14                 echo "bye"
15                 fi
16             while [ $remainder -ne 0 ]
17             do
18                 remainder=$((n1%n2))
19                 n1=$n2
20                 n2=$remainder
21             done
22
23             gcd $1 $2
24             echo "GCD is $n1"
25         }
26 done

```

[Решить снова](#)

Рисунок 21: Решение НОД

Выполнение 3.4. Скрипты на bash: разное

3.4 Скрипты на bash: разное 10 из 10 шагов пройдено 14 из 14 баллов получено

Напишите **калькулятор** на **bash**. При запуске ваш скрипт должен ожидать ввода пользователем команды (при этом на экран выводить ничего не нужно). Команды могут быть трех типов:

1. Слово **"exit"** в этом случае скрипт должен вывести на экран слово **"bye"** и завершить работу.
2. Три аргумента **через пробел** – первый операнд (целое число), операция (одна из "+", "-", "*", "/", "%", "**") и второй операнд (целое число). В этом случае нужно произвести указанную операцию над заданными числами и вывести результат на экран. После этого переходим в режим ожидания новой команды.
3. Любая другая команда из одного аргумента или из трех аргументов, но с операцией не из списка. В этом случае нужно вывести на экран слово **"error"** и завершить работу.

Чтобы проверить работу скрипта, вы можете записать сразу несколько команд в файл и передать его скрипту на stdin (т.е. выполнить `./script.sh < input.txt`). В этом случае он должен вывести сразу все ответы на экран.

Например, если входной файл будет следующего содержания:

```
10 + 1
2 ** 10
exit
```

то на экране будет:

```
11
1024
bye
```

Если же на вход поступит следующий файл:

```
3 - 5
2/10
exit
```

то на экране будет:

-2
error

т.к. вторая команда была **некорректной** (в ней всего один аргумент, т.к. нет пробелов между числами и операцией, а единственная допустимая команда из одного аргумента это "exit").

Подсказка: в случае проблем с решением задачи, обратите внимание [на наши рекомендации по написанию скриптов](#).

Рисунок 22: Задание на калькулятор

Выполнение 3.4. Скрипты на bash: разное

Напишите программу. Тестируется через stdin → stdout



Отлично!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2
3 while [[ True ]]
4 do
5     read num1 op num2
6     if [[ $num1 == "exit" ]]
7     then
8         echo "bye"
9         break
10    elif [[ *$num1* =~ ^[0-9]+$ && *$num2* =~ ^[0-9]+$ ]]
11    then
12        echo "error"
13        break
14    else
15        case $op in
16            "+") let "res = num1 + num2";;
17            "-") let "res = num1 - num2";;
18            "/" ) let "res = num1 / num2";;
19            "*" ) let "res = num1 * num2";;
20            "%" ) let "res = num1 % num2";;
21            "**") let "res = num1 ** num2";;
22            *) echo "error" ; exit ;;
23        esac
24        echo "$res"
25    fi
26 done
```

Следующий шаг

Решить снова

Рисунок 23: Решение калькулятора

Выполнение 3.5. Продвинутый поиск и редактирование

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Пусть в директории `/home/b1` лежат файлы `Star_Wars.avi`, `star_trek OST.mp3`, `STARS.txt`, `stardust.mpeg`, `Eddard_Stark_biography.txt`.

Отметьте все файлы, которые **найдет** команда `find /home/b1 -iname "star*"`, но **НЕ найдет** команда `find /home/b1 -name "star*"`?

Выберите все подходящие ответы из списка

✔ Отличное решение!

- ☐ stardust.mpeg
- ☐ star_trek OST.mp3
- ☒ Star_Wars.avi
- ☒ STARS.txt
- ☐ Eddard_Stark_biography.txt

Следующий шаг Решить снова

Вы получили: 1 балл
[Мои решения](#)

Верно решили 30 903 учащихся
Из всех попыток 40% верных

Рисунок 24: find с -iname и -name

Выполнение 3.5. Продвинутый поиск и редактирование

3.5 Продвину́тый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте все верные утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

✓ Хорошая работа.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форме решений](#).

- ☐ Опция -path и -name всегда работают одинаково
- ☐ Опция -path используется только для поиска директорий, а -name только для поиска файлов
- ☒ В некоторых случаях find с -name найдет меньше файлов, чем find с таким же запросом, но с -path
- ☐ Опция -path аналогична -name, но игнорирует размер букв (строчные/прописные) в имени файла
- ☒ В некоторых случаях find с -name найдет больше файлов, чем find с таким же запросом, но с -path

[Решить снова](#)

Вы получили: 1 балл

Верно решили 28 288 учащихся
Из всех попыток 26% верных

Рисунок 25: Сравнение -path и -name

Выполнение 3.5. Продвинутый поиск и редактирование

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Предположим, что в директории `/home/b1/` есть следующая структура файлов и поддиректорий:

```
/home/b1/  
├── dir1  
│   ├── file1  
│   └── dir2  
│       ├── file2  
│       └── dir3  
│           └── file3
```

Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/b1 -mindepth 2 -maxdepth 3 -name "file*"`?

Выберите один вариант из списка

☒ Верно.

- ☐ Все кроме `file2`
- ☐ Только `file1`
- ☒ Все кроме `file3`
- ☐ Ни один файл найден не будет
- ☐ Только `file3`

Следующий шаг

Решить снова

Рисунок 26: `mindepth` и `maxdepth` в `find`

Выполнение 3.5. Продвинутый поиск и редактирование

3.5 Продвину́тый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем в каждой строке есть слово `"word"`. Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

То какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один вариант из списка

Отлично!

- ☐ Bce, xпомe grep "word" file.txt > results.txt
- ☐ grep -C 1 "word" file.txt > results.txt
- ☒ results.txt будет одинакового размера во всех случаях
- ☐ grep -A 1 "word" file.txt > results.txt
- ☐ grep -A 1 "word" file.txt > results.txt и grep -B 1 "word" file.txt > results.txt

[Решить снова](#)

Рисунок 27: сравнение размеров вывода `grep` с контекстом

Выполнение 3.5. Продвинутый поиск и редактирование

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E '[xkXkL]?[uU]buntu$' text.txt`.

Выберите все подходящие ответы из списка

✔ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ I prefer Kubuntu
- ☐ Ubuntu is better than Windows
- ☐ Well, xubuntu is OK
- ☐ Kubuntu
- ☐ Mac OS X 10.9, Windows XP, Ubuntu 12.04
- ☐ Uuuubuntu!

[Решить снова](#)

Рисунок 28: grep с регулярным выражением

Выполнение 3.5. Продвинутый поиск и редактирование

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Что произойдет, если в команде `sed -n "/[a-z]*p" text.txt` не указывать опцию `-n` ?

Выберите один вариант из списка

✔ Отлично!

- ☒ Каждая строчка будет выведена два раза
- ☐ Будут выведены все строки файла `text.txt`, в которых есть только большие буквы латинского алфавита
- ☐ На экран будет выведено всё содержимое файла `text.txt`
- ☐ На экран ничего не напечатается

[Следующий шаг](#) [Решить снова](#)

Вы получили: 1 балл
[Мои решения](#)

Верно решил 29 891 учащихся
Из всех попыток 40% верных

3 учащимся понравился этот шаг, а вам?
😊 😊 😊 😊 😊

[Следующий шаг >](#)

Рисунок 29: sed без -n

Выполнение 3.5. Продвинутый поиск и редактирование

3.5 Продвину́тый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Запишите в форму ниже инструкцию `sed`, которая заменит все "аббревиатуры" в файле `input.txt` на слово "abbreviation" и запишет результат в файл `edited.txt` (на экран при этом ничего выводить не нужно). Обратите внимание, что в инструкции должны быть указаны и сам `sed`, и оба файла!

Под "аббревиатурой" будем понимать слово, которое удовлетворяет следующим условиям:

- состоит только из больших букв латинского алфавита,
- состоит из хотя бы двух букв,
- окружено одним пробелом с каждой стороны.

При этом будем считать, что в тексте **не может быть две "аббревиатуры" подряд**. Например, текст "YOU YOU and YOU!" является **некорректным** (в нем есть две "аббревиатуры", но они идут подряд) и на таких примерах мы проверять вашу инструкцию **не будем**.

Пример: если у вас был текст "Hi, I heard these songs by ABBA, TLA and DM !", то он должен быть преобразован в "Hi, I heard these songs by ABBA, abbreviation and abbreviation !".

Примечание: после вашей замены "аббревиатуры" на слово "abbreviation" количество пробелов в тексте не должно меняться!

Внимание! Во время проверки мы не запускаем команду, которую вы ввели на реальном файле с «аббревиатурами» (это небезопасно, можно же ввести `rm -rf *`!). Вместо этого мы сперва анализируем структуру ваших инструкций (например, что в ней использовано именно `sed` и сделано это ровно один раз, что на вход подается `input.txt`, а результат будет записан в `edited.txt` и т.д.), а затем **запускаем её с смысловую часть** (т.е. поиск по регулярному выражению и замена на "abbreviation") на тестовых примерах. К сожалению, наш запуск не идеально повторяет `sed`, но он очень близок к нему. Главная «несовместимость» заключается в том, что наша проверка не понимает идущие подряд символы, отвечающие за количество повторений (т.е. *, ?, ? и !). Однако эту «несовместимость» легко исправить указав при помощи «|» какой из символов к чему относится! Например, регулярное выражение `a+?` (ноль или один раз по одной или более букве "a") нужно записать как `(a+)?` (при этом запись `(a)+?`, конечно же, не поможет).

Рисунок 30: Замена аббревиатур – условие

Выполнение 3.5. Продвинутый поиск и редактирование

3.5 Продвинутый поиск и редактирование 13 из 13 шагов пройдено 10 из 10 баллов получено

Запишите в форму ниже инструкцию `sed`, которая заменит все "аббревиатуры" в файле `input.txt` на слово "abbreviation" и запишет результат в файл `edited.txt` (на экран при этом ничего выводить не нужно). Обратите внимание, что **в инструкции должны быть указаны** и сам `sed`, и обо файлах!

Под "аббревиатурой" будем понимать слово, которое удовлетворяет следующим условиям:

- состоит только из больших букв латинского алфавита,
- состоит из хотя бы двух букв,
- окружено одним пробелом с каждой стороны.

При этом будем считать, что в тексте **не может быть две "аббревиатуры" подряд**. Например, текст `" YOU YOU and YOU!"` является **некорректным** (в нем есть две "аббревиатуры", но они идут подряд) и на таких примерах мы проверять вашу инструкцию **не будем**.

Пример: если у вас был текст `"Hi, I heard these songs by ABBA, TLK and BR !"`, то он должен быть преобразован в `"Hi, I heard these songs by ABBA, abbreviation and abbreviation !"`.

Примечание: после вашей замены "аббревиатуры" на слово "abbreviation" **количество пробелов в тексте не должно меняться!**

Внимание! Во время проверки **мы не запускаем команду**, которую вы ввели на реальном файле с "аббревиатурами" (это небезопасно, можно же ввести `rm -rf /*`!) Вместо этого мы сперва анализируем структуру вашей инструкции (например, что в ней использованы именно `sed` и сделано это ровно один раз, что на вход подается `input.txt`, а результат будет записан в `edited.txt` и т.д.), а затем **запускаем её** на тестовую часть (т.е. поиск по регулярному выражению и замена на "abbreviation") на тестовых примерах. К сожалению, наш запуск не идеально повторяет `sed`, но он очень близок к нему. Главная "несовместимость" заключается в том, что наша проверка не понимает идущие подряд символы, отвечающие за количество повторений (т.е. `*`, `+`, `?` и `{}`). Однако эту "несовместимость" легко исправить указав при помощи `[]` и `?` какой из символов к чему относится! Например, регулярное выражение `[a+]` (ноль или один раз по одной или более букве "a") нужно записать как `[a+]?` (при этом запись `[a+]??`, конечно же, не поможет).

Напишите текст

Верно.

Следующий шагРешить снова

Рисунок 31: Замена аббревиатур – решение

Выполнение 3.6. Строим графики в gnuplot

3.6 Строим графики в gnuplot 10 из 10 шагов пройдено 7 из 7 баллов получено

Вы можете скачать и попробовать применить gnuplot к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске gnuplot, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

✓ Прекрасный ответ.

- ☒ -p, --persist
- ☐ Графики и так не закрываются автоматически при закрытии gnuplot!
- ☐ Такой опции не существует
- ☐ -raise

Следующий шаг

Решить снова

Рисунок 32: Сохранение графиков после закрытия gnuplot

Выполнение 3.6. Строим графики в gnuplot

3.6 Строим графики в gnuplot 10 из 10 шагов пройдено 7 из 7 баллов получено

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете gnuplot и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет **название** у построенного **ряда данных** и **сколько** будет нарисовано **точек** на графике?

Выберите один вариант из списка

☒ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☐ Название "попате", нарисовано 10 точек
- ☐ Название – первое значение из первого столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☐ Название – первое значение из второго столбца, нарисовано 10 точек
- ☐ Название "data.csv using 1:2", нарисовано 10 точек

Следующий шаг

Решить снова

Вы получили: 1 балл

[Мои решения](#)

Верно решили **27 489** учащихся

Из всех попыток **34%** верных

Рисунок 33: autotitle columnhead без заголовка

Выполнение 3.6. Строим графики в gnuplot

3.6 Строим графики в opusplot 10 из 10 шагов пройдено 7 из 7 баллов, получено

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [plot.gnu](#), [plot_advanced.gnu](#), [plot_advanced2.gnu](#). Все три скрипта основаны на [этой заметке](#), данные также взяты оттуда.

Предполагая, что вы пишете `plot`-скрипт и у вас в нем есть три переменные `x1`, `x2`, `x3`, в которых записаны координаты различных точек по оси `X` (по возрастанию). Вы хотите, чтобы на этой оси было только три деления (т.е. три черточки) и эти самых координатах, а подлисы этих делений были оформлены в виде **"point #номер точки, value #значение соответствующей переменной"**. Например, для `x1=8`, `x2=18`, `x3=28`, это были бы надписи **"point 1, value 0"** в точке с координатой 0 по горизонтали, **"point 2, value 10"** в точке с координатой 10 и **"point 3, value 20"** в точке с координатой 20. Или, например, `x1=180`, `x2=150`, `x3=250`, это были бы надписи **"point 1, value 100"** в точке с координатой 100, **"point 2, value 150"** в точке с координатой 150 и **"point 3, value 250"** в точке с координатой 250.

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно дописать в скрипт, для выполнения этой задачи.

Примечание: проверять, что переменные `x1`, `x2`, `x3` идут по возрастанию или что они являются числами не нужно!

Примечание 2: в видеофрагменте на предыдущем шаге звучал термин конкатенация, который важен для выполнения данного задания. Под конкатенацией обычно понимают "склеивание" двух строк в одну длинную строку, например, конкатенация строк "Данные из файла " и "data.csv" даст строку "Данные из файла data.csv".

Подсказка: настоятельно рекомендуем изучить примеры скриптов – в них есть большая часть решения!

Напишите текст

☒ Верно.

```
set xtics ("point 1, value ".x1 x1, "point 2, value ".x2 x2, "point 3, value ".x3 x3)
```

[Решить снова](#)

Рисунок 34: Установка пользовательских xtics

Выполнение 3.6. Строим графики в gnuplot

3.6 Строим графики в gnuplot 10 из 10 шагов пройдено 7 из 7 баллов получено

Если вы не скачали на предыдущем шаге файлы [animated.gnu](#) и [move.rot](#), то скачайте их теперь, т.к. они понадобятся для выполнения задания.

Указанные файлы использовались в последнем видеофрагменте для создания вращающегося графика. Измените инструкции в файле `move.rot` (т.е. **добавлять** и **удалять** инструкции **нельзя!**) таким образом, чтобы:

- График **отразился зеркально** относительно горизонтальной поверхности. То есть там, где была точка (10, 10, 200), станет точка (10, 10, -200), где была точка (-10, -10, 200) станет (-10, -10, -200) и т.д. При этом точка (0, 0, 0) останется на месте.
- Изображение стало **вращаться в обратную сторону**. То есть если раньше вращалось "влево", то теперь станет "вправо".
- Вращение стало **в два раза быстрее**. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Измененный файл загрузите в форму ниже.

Примечание: наша система проверки **не может** запустить на вашем файле `move.rot` программу gnuplot и сравнить полученный график с заданным. Вместо этого **мы анализируем команды**, которые вы указали в файле. Поэтому если вы видите, что ваш скрипт в gnuplot работает точно по условию, а мы отвечаем "Incorrect/Неверно", то попробуйте упростить свою модификацию `move.rot` и отправить его еще раз.

Напишите текст

✓ Всё получилось!

```
a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot x**2-y**2
pause 0.1
if (a<50) reread
```

Следующий шаг

Решить снова

Рисунок 35: Модифицированный move.rot

Выполнение 3.7. Разное

3.7 Разное 15 из 15 шагов пройдено 7 из 7 баллов получено

Какая команда(ы) установят файлу `file.txt` права доступа `rw-rw-r--`, если изначально у него были права `r--r--r--`. Укажите все верные варианты ответа!

Примечание: запись вида команда1; команда2; команда3 означает, что в терминале последовательно выполнялись все три команды (сначала команда1, затем команда2 и, наконец, команда3).

Выберите все подходящие ответы из списка

✔ Абсолютно точно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ chmod u+wx file.txt; chmod g+w file.txt
- ☒ chmod ug+w file.txt; chmod u+x file.txt
- ☐ chmod u-wx file.txt; chmod g-w file.txt
- ☐ chmod 764 file.txt
- ☒ chmod a+wx file.txt; chmod o-wx file.txt; chmod g-x file.txt
- ☐ chmod o-wx file.txt; chmod g-x file.txt; chmod a+wx file.txt

[Решить снова](#)

Вы получили: 1 балл

Мои решения

Верно решили 24 989 учащихся

Из всех попыток **21%** верных

Рисунок 36: Изменение прав доступа

Выполнение 3.7. Разное

3.7 Разное 15 из 15 шагов пройдено 7 из 7 баллов получено

Предположим вы использовали команду `sudo` для создания директории `dir`. По умолчанию для `dir` были выставлены права доступа `rw-r-x-r-x` (владелец `root`, группа `root`). Таким образом никто кроме пользователя `root` не может ничего записывать в эту директорию, например, не может создавать файлы в ней.

После выполнения какой команды **user** из группы **group** всё-таки сможет создать файл внутри **dir** ? Укажите **все верные** варианты ответов!

Примечание: считаем, что все команды выполняются от имени `user`, если явно не указано, что команда выполнена с `sudo`.

Примечание 2: мы выбрали пример с директорией, а не с файлом не случайно.

Дело в том, что если создать при помощи `sudo` файл с правами `rw-r--r--` в директории, которая принадлежит пользователю, то возникает любопытная ситуация. Стороны пользователь может удалить этот файл (т.к. ему разрешено удалять **все** файлы внутри его директории) и может прочитать его содержимое (т.к. право `rw` у файла установлено для всех), с другой стороны он не может этот файл редактировать (т.к. право `w` у файла есть только для `root`). При этом некоторые "умные" редакторы, например, `vim` позволяют даже редактировать этот файл, но делают они это своеобразно: через удаление оригинала и создание копии уже с нужными правами (удалить мы можем, а раз можем читать, то и копию создать не сложно). Итого получается, что несмотря на права `rw-r--r--`, пользователь может сделать с этим файлом почти всё что угодно!

В случае же, когда речь идет о директории созданной **root**, ситуация будет проще: пользователь сможет смотреть её содержимое (у него есть право **r**), но удалять и создавать файлы в ней не сможет (права **w** у него нет).

Важно отметить, что директория в Linux это в каком-то смысле файлы. Содержимое такого "файла" – это записи о файлах и поддиректориях этой директории (грубо говоря их название). Таким образом, право "r" у директории дает возможность просматривать "записи", т.е. просматривать их состав. Право "w" у директории дает возможность удалять/добавлять новые "записи", т.е. удалять/создавать файлы/поддиректории в ней.

На самом деле и это еще не всё. Существует так называемый **sticky bit** (атрибут файла или директории), выставление которого меняет описанное выше поведение. Файлы (или директории) с таким атрибутом сможет удалить только их владелец вне зависимости от прав, установленных у директории, в которой эти файлы (или директории) лежат!

Отдельное спасибо слушателю курса **Alexey Antipovsky** за помощь в оформлении Примечания 2!

Выберите все подходящие ответы из списка

✔ Отличное решение!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ sudo chmod o+w dir
- ☒ sudo chown user:group dir
- ☒ sudo chmod a+w dir
- ☐ chmod o+w dir
- ☐ sudo chmod o+x dir
- ☒ sudo chown user dir

Выполнение 3.7. Разное

3.7 Разное 15 из 15 шагов пройдено 7 из 7 баллов получено

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`

Выберите все подходящие ответы из списка

✓ Хорошая работа.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариих](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Количество определенных букв (например, количество букв "А")
- ☒ Количество символов
- ☒ Количество строк
- ☒ Длину самой длинной строки
- ☒ Количество слов

[Решить снова](#)

Рисунок 38: Возможности ws

Выполнение 3.7. Разное

3.7 Разное 15 из 15 шагов пройдено 7 из 7 баллов получено

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести в **удобном для чтения формате** (например, вместо 2048 байт надо вывести 2.0К) и **больше** на экран выводить **ничего не нужно**). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по 800 Кбайт и две поддиректории в каждой из которой лежит по файлу в 400 Кбайт, то загаданная команда должна вывести на экран одно число: 2.4К (также на экране может быть выведен еще и символ "", обозначающий, что это размер именно текущей директории).

Напишите текст

✔ Отлично!

```
du -h -s
```

Следующий шаг

Решить снова

Рисунок 39: Вывод размера директории

Выполнение 3.7. Разное

3.7 Разное 15 из 15 шагов пройдено 7 из 7 баллов получено

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполнит эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст

✔ Правильно.

```
mkdir dir{1..3}
```

Следующий шаг

Решить снова

Рисунок 40: Кратчайшее создание нескольких каталогов

Section 3

Результаты

В ходе выполнения третьего этапа курса «Введение в Linux» были освоены продвинутые приёмы работы в vim, написание сценариев на bash с условными переходами и циклами, использование функций, регулярные выражения в grep и sed, построение и анимация графиков в gnuplot, а также ряд полезных утилит (wc, du, chmod)

- Курс «Введение в Linux» на платформе Stepik [Электронный ресурс] URL: <https://stepik.org/course/73/>